1. **How Machine Understands Text Data:**

They convert text into numbers (vectors) and then apply algorithms.

Step 2: Text Preprocessing (Cleaning):

- **This prepares text for modeling.**
- **Lowercasing**
- **Remove punctuation**
- **Tokenization(split into words)**

Step 3: Convert Text → Numbers (Vectorization)

Common techniques:

1. Bag of Words (BoW)
    - Counts word frequency
2. TF-IDF (Term Frequency – Inverse Document Frequency)
    - Importance of words in a document
3. Word Embeddings (Advanced)
    - Words converted into dense vectors
    - Captures meaning

Step 4: Model Understanding

Once text becomes numbers, ML models process it:

- Logistic Regression → classification
- Naive Bayes → spam detection
- Deep Learning → sentiment analysis, chatbots

NLP (Natural Language Processing)

- Deals with processing text

Tasks:

- Tokenization
- POS tagging
- Named Entity Recognition (NER)
- Translation

NLU (Natural Language Understanding)

- Subset of NLP
- Focus: Meaning + context

Tasks:

- Intent detection
- Sentiment analysis
- Chatbot understanding

NLP Toolkits:

Python Libraries

NLTK (Natural Language Toolkit)

- Basic NLP tasks
- Tokenization, stemming

spaCy

- Fast and production-ready
- Named Entity Recognition (NER)
- POS tagging

TextBlob

- Easy for beginners
- Sentiment analysis

Deep Learning / Modern NLP

TensorFlow / PyTorch

- Build NLP models

Tokenization:

Tokenization = Breaking text into smaller units (tokens)
Tokens can be Words, Sentences, Characters, Sub-words
Example:
"Machine learning is powerful"
["Machine", "learning", "is", "powerful"]
It only Splits text, Prepares data

POS Tagging (Part-of-Speech Tagging):

POS Tagging = Assigning grammatical labels to each token (word)
It tells what role each word plays in a sentence
Understanding sentence structure
Improves NLP tasks accuracy
Used in:

- Chatbots (intent understanding)
- Grammar correction
- Named Entity Recognition (NER)
- Machine translation

Stop Words:

Common words that carry little meaningful information.

- Exp: the, is, in, at, on, a, an

Why Remove Stop Words?

- Reduce noise
- Reduce dimensionality
- Improve model efficiency

Stemming:

Both are used to normalize words → convert to base/root form
Cutting words to root form by removing suffix/prefix.
Works by truncation.

Studies → Studi

Running → Run

Snowball Stemmer: Improved version of Porter Stemmer

Lemmatization: Converting word to its base (dictionary form) using vocabulary + grammar.

Key Characteristics

- Uses dictionary (WordNet)
- Context-aware
- Produces valid words

Limitation

- Slower than stemming
- Requires POS tagging for best results

Stemming = "Cut the word"
Lemmatization = "Understand and convert the word"

Named Entity Recognition (NER):
Identifying and classifying important entities in text into predefined categories.
It does not tag every word, only specific meaningful entities.

PERSON → names
ORGANIZATION → companies
LOCATION → cities, countries

NER = **information extraction**

Parsing: Analyzing grammatical structure of a sentence.

- It shows how words are related
- Breaks sentence into phrases

"He eats an apple"
eats → main verb
He → subject
apple → object

Why Parsing is Important

- Sentence understanding
- Question answering
- Machine translation
- Grammar checking

Word Embedding:
Converting words into dense numerical vectors that capture meaning and relationships.
Not just numbers — **meaningful numbers**
Similar words → similar vectors
Words with similar meaning are closer in vector space

Why Embeddings are Important

- Capture meaning (semantics)
- Improve model performance

- Used in:
    - Chatbots
    - Search engines
    - Recommendation systems

One-Hot Encoding:
Represents each word as a binary vector (0s and 1s)
Each word = unique index in vocabulary
Limitations
- No meaning captured
- Sparse vectors (mostly 0s)
- No relationship between words

Bag of Words (BoW)
Counts frequency of words in a document
Ignores grammar and order.
BoW = **Term Frequency (TF)**
Limitations
- No word order
- No context
- Important words may be over-weighted

"I love AI AI"

["I", "love", "AI"]

[1, 1, 2]

N-Gram Model:
Captures sequence of words (order matters)
Unlike BoW, it keeps context

Types of N-grams:
- **Unigram (1 word)**
  "I love AI"
  ["I", "love", "AI"]
- **Bigram (2 words)**
  ["I love", "love AI"]
- **Trigram (3 words)**
  ["I love AI"]

Skip-Gram
Allows skipping words to capture wider context
"I love AI very much"
("I", "AI"), ("love", "very")

Capture context and order
Useful in:
- Language modeling
- Text prediction

- Speech recognition

Term Frequency (TF):
Term Frequency (TF) measures how frequently a word (term) appears in a document relative to the total number of words in that document.

TF(t)=(Number of times term t appears in document / Total number of terms in document)

frequency of words appears in document. vs total number of words in document.

- TF is always between 0 and 1.
- TF is calculated **per document**
- Same word can have different TF in different documents
- Higher TF = word is more important within that document

Limitations of TF:
"the", "is", "and" → high TF but low meaning

Inverse Document Frequency (IDF):
IDF measures how important (rare) a word is across all documents
Words that appear in many documents → less important
Words that appear in few documents → more important